Expert Systems With Applications 331 (2026) 133229

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

UDfSolver : Uniform representation and decoupled inference strategy for text-diagram function problems solving

Feng Xiong, Xinguo Yu *, Litian Huang, Dian Yu

National Engineering Research Center for E-Learning, Faculty of Artificial Intelligence in Education, Central China Normal University, 152 Luoyu Road, Hongshan District, Wuhan, 430079, China

ARTICLE INFO

Keywords:
Function problem solving
Uniform representation
Symbolic solver
Decoupled inference strategy
Meta-model inference

ABSTRACT

Developing generalized algorithms for solving diverse Text-Diagram Function (TDF) problems is challenging due to the high variability in function types and underlying mathematical structures. Existing methods often fail to adapt across different TDF problem categories, limiting their applicability. This paper introduces UDfSolver, a novel framework designed to address six distinct TDF problem categories: linear, quadratic, inverse proportional, exponential, logarithmic, and power functions. UDfSolver incorporates two key innovations: (1) a Uniform Representation Model (URM) for standardized problem recognizance and outcome representation, and (2) a Decoupled Inference Strategy (DIS) that separates procedural inference from type-specific reasoning, enabling a two-layer inference process. URM unifies the representation of different function problem. DIS enhances modularity and scalability by decoupling the steps of problem interpretation and symbolic solution generation. Extensive evaluations across six TDF categories and three benchmark datasets demonstrate that UDfSolver significantly outperforms existing methods in accuracy, generality, and interpretability. The framework provides a robust and scalable solution for diverse TDF problems, advancing the state of the art in mathematical problem-solving.

1. Introduction

Developing a generalized algorithm for solving diverse Text-Diagram Function (TDF) problems is a critical challenge in both theoretical and applied domains. In theory, it represents a pivotal step toward constructing compact machines capable of integrating high-performance solving algorithms to address mathematical problems (Yu et al., 2026). In practice, such algorithms are foundational technologies for advanced intelligent education systems (Lin et al., 2023; Lu et al., 2021; Yu et al., 2022). However, the wide variability in function types–including linear, quadratic, inverse proportional, exponential, logarithmic, and power functions–each with distinct mathematical structures and graphical forms, has hindered progress (Sokolowski, 2024). Existing methods often adopt a fragmented approach, where specialized algorithms are designed for single function types, failing to adapt across different function types (Huang et al., 2021; Wilkie, 2024; Yin et al., 2019; Yu et al., 2022). Moreover, these specialized algorithms ignore the core educational goal of understanding the essence of the function concept from diverse function types (Hatisaru & Erbas, 2017). This limitation underscores the critical need for a unified framework that can consistently represent and solve diverse TDF problems.

A Text-Diagram Function (TDF) problem involves both text descriptions and function diagrams, presenting core challenges rooted in two key aspects: the diversity of function types and the composite nature of functions. The inherent heterogeneity of function types, such as the constant rate of change in linear functions or the parabolic forms of quadratic functions, introduces significant complexity (Sokolowski, 2024). Additionally, functions are inherently composite entities, comprising independent and dependent variables, type-specific characteristics (e.g., slope, curvature), and domain constraints. This multi-faceted structure cannot be captured by a single variable, further complicating the development of a generalized solving framework. It is noteworthy that this work focuses on TDF problems of explicit function knowledge acquisition and inference. These problems examine the understanding of formal function concepts, function properties, point-intersection relations and symbolic expressions. The current framework does not address the TDF problems that require the extraction of implicit function knowledge from real-world or physical scenarios.

* Corresponding author.
E-mail addresses: mountain@mails.ccnu.edu.cn (F. Xiong), xgyu@ccnu.edu.cn (X. Yu), litianhuang@mails.ccnu.edu.cn (L. Huang), yudian@mails.ccnu.edu.cn (D. Yu).

https://doi.org/10.1016/j.eswa.2026.133229
Received 31 December 2025; Received in revised form 26 May 2026; Accepted 8 June 2026
Available online 23 June 2026

Existing research are primarily divided into relation-flow approaches (Peng et al., 2025; Yu et al., 2023, 2022) and multimodal large language models (MLLMs) (Wang et al., 2025; Zhang et al., 2024a). Relation-flow approaches are highly specialized methods to solve math problems. For instance, Yu et al. (2022) built a relation-centric algorithms for solving piece-wise linear text-diagram function problems. However, these solutions are largely confined to single function type, lacking adaptability across diverse function types. Conversely, while MLLMs (Lu et al., 2024; Zhang et al., 2024c) demonstrate broad coverage across general benchmarks, their problem-solving accuracy remains insufficient. Furthermore, MLLMs also suffer from hallucination issues and lack precision in processing strict function properties, making them unreliable for stable application in educational scenarios.

This paper introduces UDfSolver (Uniform Representation and Decoupled Inference Strategy function Solver), a novel generalized framework designed to address six distinct TDF problem types: linear, quadratic, inverse proportional, exponential, logarithmic, and power functions. UDfSolver integrates two key innovations: (1) a Uniform Representation Model (URM) for standardized problem Recognizance and outcome representation, and (2) a Decoupled Inference Strategy (DIS) that separates procedural inference from type-specific reasoning, enabling a two-layer inference process. DIS enhances modularity and scalability by decoupling the procedural steps from type-specific reasoning. By DIS, UDfSolver maintains a consistent solving workflow while correctly handling the distinct mathematical properties of each function type.

UDfSolver is evaluated across six distinct function types–linear, quadratic, inverse proportional, exponential, logarithmic, and power functions–on three comprehensive benchmarks. The results revealed a marked enhancement over existing baseline methods, with UDfSolver achieving superior performance in accuracy, generality, and interpretability. By providing a scalable and robust solution, this framework represents a significant leap forward in the domain, setting the stage for more efficient and adaptable intelligent systems capable of addressing complex function-based challenges.

The contributions of this paper are as follows:

1. A UDfSolver is built to solve six types of TDF problems, which are linear, quadratic, inverse proportional, exponential, logarithmic, and power functions.
2. A uniform representation model (URM) is proposed to provide a structured and consistent format for encoding knowledge from both text and diagrams across six types of functions.
3. A decoupled inference strategy (DIS) is innovated to decouple the entire inference into the procedural inference and function metamodel inference.

The remainder of this paper is structured as follows: Section 2 reviews the related work in the field. Section 3 formalizes the problem statement and provides an overview of UDfSolver. Section 4 introduces the uniform representation model (URM), detailing its construction and how it accommodates diverse function types. Section 5 presents the decoupled inference strategy (DIS) and elaborates on its step-by-step inference procedure. Section 6 uses an example to illustrate the workflow of UDfSolver. Section 7 evaluates UDfSolver through experimental analysis. Section 8 discusses the generalization and scalability of UDfSolver. Section 9 concludes the paper with key findings and future directions.

2. Related work

This section starts with a review of existing approaches for solving TDF problems. The algorithm proposed in this paper consists of two stages: problem recognizance and symbolic solving. Therefore, the subsequent content will also review the related work in function problem recognizance and symbolic solver.

2.1. Function problem solving

Research in function problem solving can be divided into two approaches: (1) relation-flow and (2) LLM-based.

Relation-flow Approach is a series of problem-solving methods that has been developed to solve arithmetic word problems (Yu et al., 2026, 2023, 2019) and geometry problems (Gan et al., 2019; Huang et al., 2023). The relation-flow approach utilizes relations as an intermediate state, and it operates on two phases to form equations and solve the problem. The first phase aims to extract a group of relations; the second phase transforms the relations into a system of equations, then solved by the exiting math method. Inspired by the success of these algorithms, Yu et al. (2022) extended the relation-flow approach to develop the algorithm for solving piece-wise linear TDF problems. However, existing relation-flow algorithms are largely confined to linear types. They lack a unified framework to handle the mathematical diversity of non-linear functions (e.g., quadratic, exponential, logarithmic) and the structural complexity of function constraints. This paper aims to bridge this gap by extending the relation-flow path to a comprehensive set of function types.

LLM-based Approach. With the advent of multimodal large language models (MLLMs) like GPT-4V (OpenAI, 2023) and specialized models like MathVerse or MultiMath-7B (Peng et al., 2024; Zhang et al., 2024c), end-to-end problem solving has seen rapid progress. While these models demonstrate broad coverage, they are generally tested on TDFs as part of larger benchmarks rather than being optimized for them. Consequently, they often suffer from hallucinations and lack precise handling of rigorous function properties (e.g., domain constraints, asymptotic behaviors), limiting their practical utility in educational settings.

Intelligent Educational Problem Solvers. Intelligent problem solvers (IPS) designed for educational settings provide important context for our work. Prior educational IPS research emphasizes readable step-by-step solutions, human-aligned reasoning, and course-oriented knowledge organization (Nawaz, 2024; Nguyen & Do, 2017). These properties serve as a reference for the interpretability and structured reasoning transparency aimed for in UDfSolver. While our system embodies these educational-aligned properties, it is important to note that this paper focuses on the algorithmic framework and does not directly evaluate learner outcomes.

Currently, research on LLMs in TDF problem solving is primarily limited to performance evaluations involving function-related benchmarks, rather than the development of specialized algorithms. As a result, their capabilities remain distant from the precision and reliability required for practical educational applications. Conversely, the customized relation-flow approach has demonstrated robust performance across a series of mathematical problem domains. Motivated by this success, this paper continues to advance the relation-flow approach, further extending and refining existing algorithms to address the complexities of diverse TDF problems.

2.2. Text-diagram function problem recognizance

Existing relation-flow approaches regard problem recognizance as the process of acquiring relations from problem, and the outcome of problem recognizance is a group of relations. This part reviews the problem recognizance methods of some relation-flow approaches related to TDF problem, including text understanding method and diagram understanding method. In diagram understanding, there is an additional diagram parsing task. As this parsing process is not the primary focus of our algorithmic research, we briefly introduce the existing methods we have adopted in this work.

Text Understanding acquires a set of function relations from problem text. Syntax-semantics (S^2) model method has been successfully employed to parse explicit relations in arithmetic and geometric problems (Gan et al., 2019; Yu et al., 2023). For function problems, Yu et al. (2022) have proposed a new approach of text understanding, using a

S^2P (syntax-semantics with period) model method for extracting function period relations from text.

Diagram Understanding extracts function relations form diagram. It is also an important component of function problem recognizance. Several solving algorithms that use the relation-flow approach have extended the S^2 model method to understand the geometric diagram (Huang et al., 2023) and function diagram (Yu et al., 2022). Specifically, Yu et al. (2022) have proposed a L^2 (line segment with labels pattern) method for mining the linear function relations from diagram by identifying line segments. However, these methods are largely tailored to linear or piecewise linear functions, lacking the generality required to understand the diverse graphical features of non-linear functions such as quadratics or exponentials.

To overcome the limitations of existing problem recognizance methods, this paper proposes two specialized enhancements: S^2F (syntax-semantics for function) model for text understanding and F^2 (function feature) model for diagram understanding. S^2F model formalizes text parsing through deterministic syntactic pattern matching. This work constructs 52 patterns and categorized into four sets: function definition, constraint, spatial, and numerical models to capture diverse function relations. Meanwhile, F^2 model generalizes diagram understanding from simple linear segments to complex spacial features (e.g., monotonicity, convexity), utilizing a poop of 23 distinct diagram patterns to identify non-linear function characteristics. Both the S^2F and F^2 model methods operate as training-free expert rule systems.

In addition, current recognizance methods typically derive a group of relations. However, the form of relation groups lack clear structure, forcing subsequent solvers to process relations in a case-by-case manner without a concise process. Furthermore, the composite nature of functions involves intrinsic constraints–such as parameter definitions and variable domains–that are difficult to encapsulate within the relation. Therefore, this work further advances beyond relation acquisition by unifying the extracted relations into a structured form named uniform representation model (URM). This structured representation enables the consistent inference process across diverse function types.

Diagram Parsing is an upstream stage of diagram understanding. To clarify the role of diagram parsing within our framework, we introduce a terminology distinction regarding the format of the input diagram: image-version diagram and vector-version diagram. The raw problem diagram is in image-version. However, the actual input to problem understanding module is assumed to be the vector-version, which represents the diagram using structured function primitives such as points, coordinate axes, and function segments. The transition from image-version to vector-version is handled by computer vision diagram parsing module. Therefore, as diagram parsing is not the primary focus of our algorithmic research, we adopt an off-the-shelf vectorized parsing pipeline (De et al., 2017). In this pipeline, a pre-trained RetinaNet (Lin et al., 2017) object detector is directly utilized to locate textual labels and key geometric primitives (e.g., the origin O) via bounding boxes. Concurrently, the Hough transform is applied for geometric vectorization, accurately extracting straight coordinate axes and linear segments. These elements are then synergized through spatial anchoring: the system computes the spatial Euclidean distance to anchor the RetinaNet-detected text labels to the nearest Hough-extracted geometric primitives. This synergized vectorized representation forms the basis for subsequent feature extraction without requiring any domain-specific neural network training.

2.3. Symbolic solver

In algorithms adopting the relation-flow approach, the core tasks of symbolic solver are inferring on the extracted group of relations to derive a system of equations and subsequently resolving these equations to generate the final solution. Researchers have developed specialized symbolic solvers for various problem-solving domains (Jian et al., 2019; Lyu et al., 2023; Peng et al., 2025; Yu et al., 2022, 2019). For instance, Yu et al. (2022) developed a specific symbolic solver capable of generating and solving mixture relation groups through an equation-function interaction method for piecewise linear function problems.

Despite these advances, existing symbolic solvers face two significant challenges when applied to solving diverse TDF problems: the increase in inference rules and the rise in equation forms. Given the involvement of six distinct function types ranging from linear to quadratic, inverse proportional, exponential, logarithmic, and power functions, the inference process demands type-specific solving workflows, as the reasoning logic varies fundamentally across function types (e.g., determining a vertex for a quadratic function versus an asymptote for an inverse proportional function). Furthermore, the inference process inevitably generates heterogeneous forms of algebraic equations. This necessitates a strategy to dynamically construct appropriate equation forms, as different function types yield distinct algebraic equations.

To overcome these challenges, this paper proposes a decoupled inference strategy (DIS) that separates the solver into two distinct layers: a meta-model inference layer and a procedural inference layer. The meta-model inference layer encapsulates type-specific computational operations, and the procedural inference layer manages the overall inference process. This decoupled design ensures that the symbolic solver remains structurally concise while possessing the flexibility to handle the mathematical complexity of diverse TDF problems.

3. Problem formulation and algorithm overview

3.1. Problem formulation

A Text-Diagram Function (TDF) problem is defined as a pair $P = \{T, D\}$, comprising a textual description T and a function diagram D. The core task is to derive a final solution S from P.

This paper focuses on six foundational function types: linear, quadratic, inverse proportional, exponential, logarithmic, and power functions. Six mathematical expressions and their corresponding general diagrams for these six function types are summarized in Table 1.

To bridge the gap between the raw, multimodal input P and the symbolic reasoning required for a solution, we introduce the concept of problem recognizance, understood state and symbolic solver.

Definition 1 (Understood State)**.** An intermediate representation of a TDF problem is called an understood state if it contains all the structured knowledge necessary for a symbolic solver to derive the final solution S without re-examining the original problem P.

To represent this understood state in a structured manner, we introduce a *Uniform Representation Model* (URM), which provides a unified structure for encoding the multimodal knowledge across diverse function types. The formal definition of the URM structure is presented in Definition 4.

Definition 2 (Problem Recognizance)**.** Problem recognizance is the process of transferring a raw TDF problem P to its corresponding understood state, which is an instance URM.

Definition 3 (Symbolic Solver)**.** Symbolic solver is the process of inferring the solution S exclusively from the knowledge contained within the understood state.

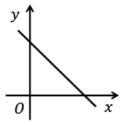
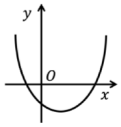
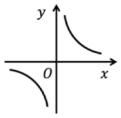
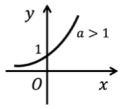
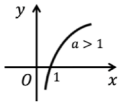
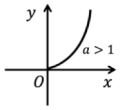
3.2. Algorithm overview

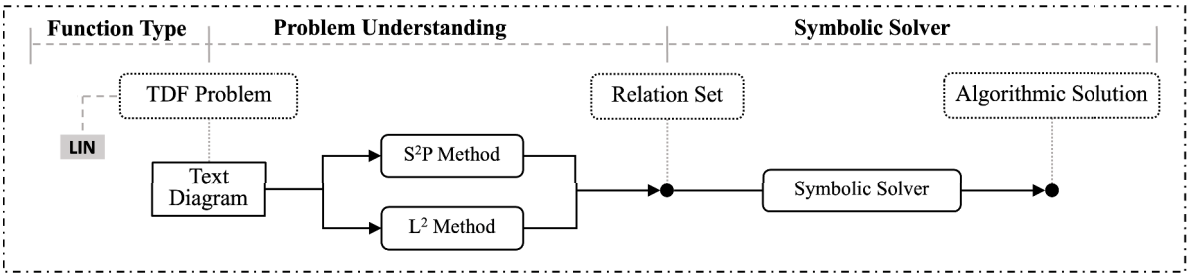
As shown in Fig. 1, the proposed algorithm is structured into two main stages, directly corresponding to the processes of problem recognizance and symbolic solver.

Stage 1: Problem Recognizance. The primary goal of this stage is to transform the raw, multimodal problem $P = \{T, D\}$ into a structured uniform representation model (URM) instance.

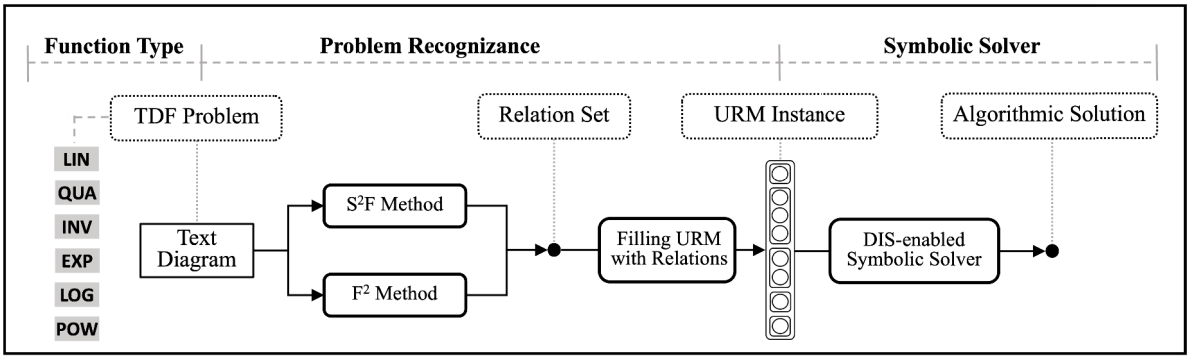
Stage 2: Symbolic Solver. In this stage, the symbolic solver operates exclusively on the URM, applying type-specific mathematical rules to the

Table 1
Mathematical expressions and their corresponding representative diagrams of six function types.

	Function Type		
	Linear	Quadratic	InverseProp
Expression	$y = kx + b$	$y = ax^2 + bx + c$	$y = \frac{k}{x}$
Diagram			
	Exponential	Logarithmic	Power
Expression	$y = a^x$	$y = \log_a x$	$y = x^a$
Diagram			



(a) Architectural components of relation-centric algorithm (Yu et al., 2022) for solving linear TDF problems.



(b) Architectural components of the proposed UDfSolver for solving six types of TDF problems.

Fig. 1. Architectural components of the proposed UDfSolver framework and its comparative relation-centric algorithm. Panel (a) depicts the component architecture of the comparative relation-centric algorithm (Yu et al., 2022) for solving linear TDF problems, whereas panel (b) outlines the component structure of the proposed UDfSolver, designed to handle six TDF problem types. Bolded components in panel (b) highlight the primary contributions: S^2F method, F^2 method, Filling URM, and DIS-enabled Symbolic Solver.

structured knowledge to construct and solve the necessary equations, ultimately deriving the final answer.

By structuring our approach in this decoupled manner, we create a system that is not only effective but also scalable and interpretable, allowing for clear analysis of both the problem recognizance and the mathematical reasoning steps. The overall architecture of this approach is shown in Fig. 1. The subsequent sections will provide a deep dive into the formalisms of the URM and the operational details of each stage (Algorithm I) .

4. Problem recognizance

This section presents methods to conduct the problem recognizance of six types of TDF problems. The novelty of this recognizance process lies in that we use a uniform representation model to represent the recognizanced result of any given problem. Besides, a S^2F (syntax-semantics for function) model method and a F^2 (function feature) model method is proposed to conduct the recognizance process.

As illustrated in Fig. 2, we decompose the problem recognizance process into three sequential steps: (1) text understanding (acquiring relations from text), (2) diagram understanding (extracting relations from diagram), and (3) filling URM with extracted relations.

4.1. Uniform representation of understood state

We first introduce and formalize the uniform representation model (URM), which serves as the understood state of a given TDF problem.

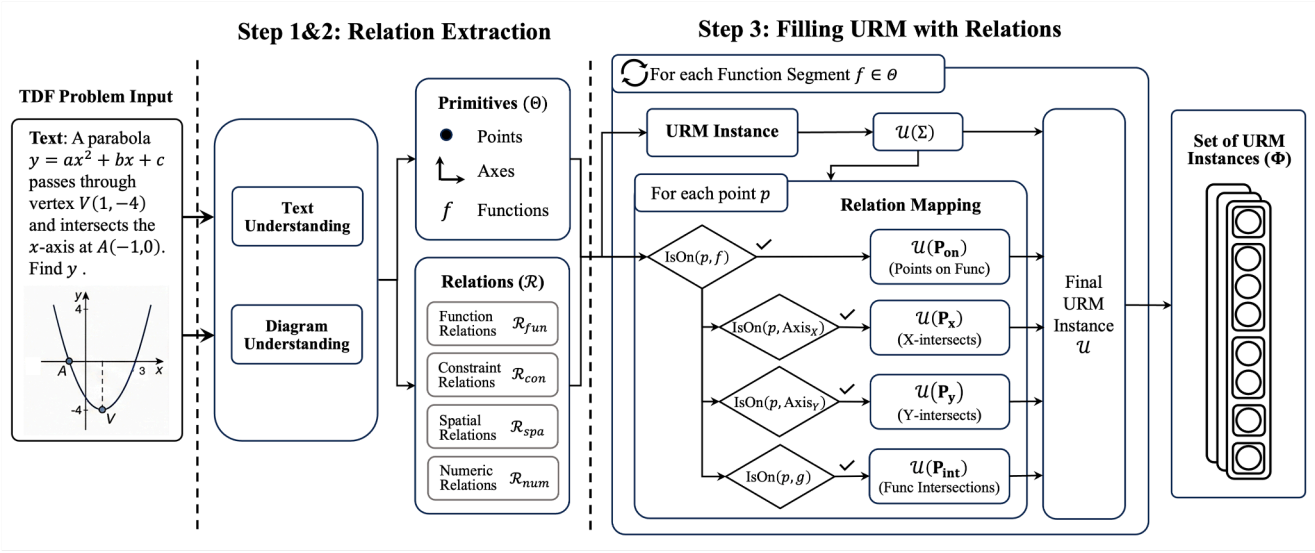


Fig. 2. Workflow of the problem recognizance process, demonstrated using a TDF problem example: a quadratic function with vertex V(1, -4) and x-intercept A(-1, 0).

Algorithm I: The Proposed Algorithm: UDfSolver.

Input: A TDF problem $P = \{T, D\}$
Output: The final solution S
Stage 1: Problem Recognizance:
begin
  Stage 1.1: Use Procedure I to acquire relation set $\mathcal{R}_T$ from text T;
  Stage 1.2: Use Procedure II to extract relation set $\mathcal{R}_D$ from diagram D;
  Stage 1.3: Use Procedure III to fill $\mathcal{R}_T$ and $\mathcal{R}_D$ into the URM instances Φ;
Stage 2: Symbolic Solver: Use Procedure IV to infer the final solution S from Φ;

The URM provides a unified structure that consistently represents six function types.

Definition 4 (Uniform Representation Model)**.** A Uniform Representation Model (URM) model is a structured representation that can represent an object of TDF, denoted as

$$\mathcal{U} = \langle \Sigma, \mathbf{P}_{on}, \mathbf{P}_x, \mathbf{P}_y, \mathbf{P}_{int} \rangle \quad (1)$$

where

- Σ encapsulates the type of the function object with its intrinsic attributes and qualitative constraints (e.g., monotonicity, vertex property, domain restrictions).
- $\mathbf{P}_{on}$ is the set of points lying on the function segment.
- $\mathbf{P}_x$ represents the set of intersection points between the function segment and the X-axis.
- $\mathbf{P}_y$ represents the set of intersection points between the function segment and the Y-axis.
- $\mathbf{P}_{int}$ indicates the set of intersection points between different function segments.

This URM model, illustrated in Fig. 3, provides a standardized structure that the symbolic solver can operate on, regardless of the original function type.

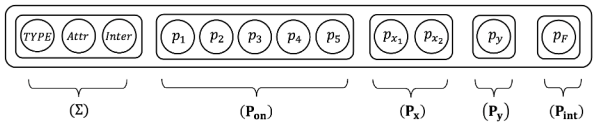


Fig. 3. Illustrating the structure of elements of uniform representation model.

4.2. Problem recognizance process

Problem recognizance process consists of three steps: (1) text understanding, (2) diagram understanding, and (3) filling URM with extracted relations. Before delving into text understanding and diagram understanding, it is crucial to preprocess the raw text and diagram separately to obtain structured tokens and primitives. For text understanding, we utilize the NLPIR-Parser (Zhang et al., 2020) and LangExtract (Goel, 2025) to segment the raw input into a token sequence labeled with Part-Of-Speech (POS) tags. This structured sequence serves as the input for the text understanding. For diagram understanding, as discussed in Section 2.2, we employ an off-the-shelf vectorized parsing pipeline (combining RetinaNet and Hough transform) to parse the raw image into discrete geometric primitives anchored with textual labels. These preprocessed structured sequences and vectorized primitives serve as the direct inputs for our proposed S^2F and F^2 models, respectively.

4.2.1. Text understanding

The primary objective of text understanding is to acquire functions relations from problem descriptions. To achieve this, we adopt the pattern matching way of the Syntax-Semantics (S^2) model method (Gan et al., 2019; Yu et al., 2023), which has been proven effective in acquiring explicit relations in arithmetic and geometric problems. We extend the S^2 model to the domain of function problems, proposing the S^2F (Syntax-Semantics for Function) model method. The S^2F model method use the syntactic pattern matching process to extract the properties and relations of the six function types in this work.

(1) Definition of S^2F models

Definition 5 (S^2F Model)**.** A Syntax-Semantics for Function (S^2F) model is a text parsing unit that formalizes the mapping from linguistic expressions to mathematical constraints. An S^2F model is defined as a

Table 2
Representative S²F models in Ω for each relation category. Each entry specifies the model structure comprising syntactic pattern ($\mathcal{K}\&\mathcal{P}$) and relation template ($\mathcal{R}$), and the examples for each listed model.

Type	S²F model		Examples	
	($\mathcal{K}\&\mathcal{P}$)	($\mathcal{R}$)	Problem Text	Extracted Relation
Function Definition	Quadratic W_f	$\{F_v, F\}$	Parabola $y = ax^2 + bx + c$	$\{F_v = [y, x], y = ax^2 + bx + c, a \neq 0\}$
	Inverse W_f	$\{F_v, F\}$	y inversely proportional to x	$\{F_v = [y, x], y = \frac{k}{x}, k \neq 0\}$
	(d n s n s)	$\{F_v, F = k \cdot s\}$	Direct proportion: price p and weight w	$\{F_v = [p, w], p = k \cdot w, w \in [0, +\infty)\}$
Constraint Relations	(s v [Prop] s)	$\{F_v, F\}$	y decreases when $x > 0$	$\{F_v = [y, x], x_1 < x_2 \Rightarrow f(x_1) > f(x_2)\}$
	(W_f [Adj])	$\{F_v, F\}$	$f(x)$ is even function	$\{F_v = [f(x), x], f(-x) = f(x), \forall x \in D\}$
Spatial Relations	(v [Prep] e)	$\{F_v, F, s\}$	Pass through point $P(x_0, y_0)$	$\{F_v = [y, x], f(x_0) = y_0, s = \text{point}\}$
	(n [Prep] W_f)	$\{F_v, F, n\}$	Axis of symmetry: $x = h$	$\{F_v = [y, x], -\frac{b}{2a} = h, n = \text{axis}\}$
Numeric Relations	(n [Prep] W_f be [Num])	$\{F_v, n, \text{val}\}$	Minimum value: $y_{\min} = v$	$\{F_v = [y, x], \min_{x \in D} f(x) = v, n = \min\}$
	([Var] > [Var])	$\{F_v, F\}$	Inequality constraint: $a > b$	$\{F_v = [a, b], a - b > 0\}$

In this table, $\mathcal{K}$ denotes keyword stems, $\mathcal{P}$ represents POS-tagged patterns: W_f = function keyword, [N]oun, [V]erb, [Adj]ective, [Prep]osition, [Num]ber, [Var]iable, [Prop] = property descriptor; $\mathcal{R}$ denotes the relation template. Symbol F represents function expression, F_v variable pairs, s primitives, n numeric descriptor, and D domain.

triplet:

$$M = \langle \mathcal{K}, \mathcal{P}, \mathcal{R} \rangle \tag{2}$$

where $\mathcal{K}$ denotes a set of keywords serving as semantic anchors (e.g., "parabola", "intercept", "increasing"); $\mathcal{P}$ represents the syntactic pattern, defined by a sequence of Parts-of-Speech (POS) tags and logical connectors; and $\mathcal{R}$ is the target relation template with unfilled slots for parameters.

(2) The pool of S²F models (Ω)

To handle the diversity of relations across the six function types, we construct the text understanding model pool as Ω. Currently, Ω comprises 52 predefined templates covering middle-school level TDF problems. This pool is categorized into four subsets corresponding to relation categories. First, function definition relation models (Ω_{fun}) identify the function type and basic algebraic structure; for instance, detecting "is a quadratic function" triggers the instantiation of the general form $y = ax^2 + bx + c$. Second, constraint relation models (Ω_{con}) extract qualitative properties such as monotonicity, parameter and coefficient constraints. A pattern like "y increases with x" maps to derivative constraints or parameter inequalities. Third, spatial relation models (Ω_{spa}) captures spatial relations between the function curve and the coordinate system, such as passing through specific points or possessing symmetry axes. Finally, numerical relation models (Ω_{num}) extract explicit quantitative assignments and inequalities such as point coordinate value. Table 2 lists representative examples of these models utilized in our system.

(3) Acquiring relations using the S²F models

The relation extraction process (Procedure I) is executed through a standard pipeline: *Tokenization → POS Tagging → Template Matching → Instantiation*.

Let T be the input problem text. The system first segments T into a sequence of tokens and assigns POS tags (e.g., Noun, Verb). It then iterates through the model pool Ω. For a candidate model $M_i \in \Omega$, if the keywords $\mathcal{K}_i$ are detected and the syntactic structure matches $\mathcal{P}_i$, the model is triggered. Finally, the specific entities in T (e.g., numbers, variable names) are extracted to fill the slots in $\mathcal{R}_i$, generating a concrete constraint instance.

Procedure I: Acquiring relations from text.

Input: Problem text T
Output: A group of relations $\mathcal{R}_T$
Load model pool Ω; $\mathcal{R}_T \leftarrow \emptyset$;
I-1: Text Parsing;
▷ Use NLPIR-Parser to transform T into $T' = \{t_1, t_2, \ldots, t_l\}$ with POS labels;
I-2: Model Matching;
▷ **for** t_i *in* T' **do**
  foreach $M = \langle \mathcal{K}, \mathcal{P}, \mathcal{R} \rangle$ *in* Ω **do**
    if *keywords* $\mathcal{K} \subseteq t_i$ ***and*** $\mathcal{P}$ *matches* t_i **then**
      Extract entities from t_i;
      Instantiate relation template $\mathcal{R}$ with extracted entities to generate r;
      $\mathcal{R}_T \leftarrow \mathcal{R}_T \cup r$;
return $\mathcal{R}_T$;

4.2.2. Diagram understanding

The primary objective of diagram understanding is to extract mathematical relations from function diagrams. To achieve this, we follow the diagramet theory (Xia & Yu, 2021) and adopt the L^2 (Line segment with Labels Pattern) model method (Yu et al., 2022), applying their segment pattern matching principles to function relation extraction. The previous L^2 model focuses on the local connectivity of line segments, which is insufficient for complex non-linear curves. Therefore, we extend the L^2 model and propose the F² (Function Feature) model method. This extension specifically identifies the diverse geometric features of the six target function types, particularly the non-linear functions.

(1) Definition of F² models

To capture the intrinsic connection between the shape of a function and its algebraic properties, we introduce the F² model.

Definition 6 (F² model). An F² model is defined as a tuple $\overline{M} = \langle \mathcal{F}, \mathcal{R} \rangle$. $\mathcal{F}$ represents the feature pattern of a function curve, encompassing spacial properties such as monotonicity, opening direction, and intercept signatures. $\mathcal{R}$ denotes the set of function relations derived from $\mathcal{F}$.

(2) The pool of F² models ($\overline{\Omega}$)

The set of F² models across the six diverse function types constitute the diagram understanding model pool denoted as $\overline{\Omega}$. The model pool $\overline{\Omega}$ contains 23 explicitly defined F² models. The fundamental principle for building this pool is to categorize the models based on the distinct geometric and spatial characteristics inherent to each function type. Table 3 lists representative examples of these F² models utilized in our system.

(3) Extracting relations using F² models

The extraction process (Procedure II) involves pattern recognition and relation instantiation. First, the diagram is parsed to identify function primitives (points, axes, curves) and anchor textual labels to them. This anchored state is then abstracted into a feature vector. The system

Table 3
Representative F^2 models in $\overline{\Omega}$ for each function type. Each entry specifies the model structure comprising feature pattern $\mathcal{F}$ and relation template $\mathcal{R}$, and the examples for each listed model.

Type	F^2 model		Examples		
	($\mathcal{F}$)	($\mathcal{R}$)	Visualization	Description	Derived Relation
Linear	Slope: +, Y-intercept: +	$\{k, b\}$		Slope: > 0, y-intercept > 0	$y = kx + b, k > 0, b > 0$
Quadratic	Opening: ↑, Axis: +, Y-intercept:-	$\{a, b, c, \Delta\}$		Opening: ↑, Axis: > 0, y-int < 0	$y = ax^2 + bx + c, a > 0, b < 0, c < 0, \Delta > 0$
Inverse Proportional	Quadrant: Q1 & Q3	$\{k\}$		Quadrant: Q1 & Q3	$y = \frac{k}{x}, k > 0$
Exponential	Trend: ↑, Y-intercept: 1	$\{a, c\}$		Trend: ↑, y-intercept (0, 1)	$y = a^x + c, a > 1, c = 0$
Logarithmic	Trend: ↑, Domain: $x > 0$	$\{a, D\}$		Trend: ↑, $D : x > 0$	$y = \log_a x, a > 1, x \in (0, +\infty)$
Power	Curvature: Convex Passes (1, 1)	$\{a\}$		Curvature: Convex	$y = x^a, a > 1$

Note: In this table, $\mathcal{F}$ denotes the feature pattern encompassing spatial properties (monotonicity, opening direction, quadrant distribution), $\mathcal{R}$ represents the relation template. Symbol k denotes slope coefficient, a, b, c function parameters, Δ discriminant, D domain.

iterates through $\overline{\Omega}$ to find a model $\overline{M}$ whose feature pattern $\mathcal{F}$ matches the diagram. Upon a successful match, the relations $\mathcal{R}$ are instantiated using the specific labels and variables from the problem context, thereby generating the relation group $\mathcal{R}_D$.

Procedure II: Extracting relations from diagram.

Input: A diagram D from P
Output: A group of relations $\mathcal{R}_D$
Load model pool $\overline{\Omega}$; $\mathcal{R}_D \leftarrow \varnothing$;
II-1: Diagram Parsing;
▷ Parse function primitives and anchor text labels in D to obtain Θ;
II-2: Feature Extraction;
▷ Construct feature vector V_{fea} from Θ;
II-3: Pattern Matching;
▷ **foreach** $\overline{M} = \langle \mathcal{F}, \mathcal{R} \rangle$ *in* $\overline{\Omega}$ **do**
  if *MatchFeature*$(V_{fea}, \mathcal{F})$ **then**
    for $\theta_i \in \Theta$ **do**
      $r' \leftarrow \text{Instantiate}(\mathcal{R}, \theta_i)$;
      $\mathcal{R}_D \leftarrow \mathcal{R}_D \cup r'$;
    break
return $\mathcal{R}_D$;

4.2.3. Filling URM with extracted relations

This step integrates the heterogeneous relations acquired from text and diagrams into the structured components of URM through three sequential actions. The URM instantiation process (Procedure III) is formalized as a mapping process $\Psi : \mathcal{R}_{all} \to \mathcal{U}$, where $\mathcal{R}_{all} = \mathcal{R}_T \cup \mathcal{R}_D$.

(1) Action I: Mapping $\mathcal{R}_{fun}$ and $\mathcal{R}_{con}$.

The process begins by establishing the object and basic structure of the function. The system maps function definition relations ($\mathcal{R}_{fun}$) to initialize the function object Σ, and also maps qualitative constraint relations ($\mathcal{R}_{con}$) into the this part.

$$\begin{aligned} \mathcal{U}.\Sigma &\leftarrow \{\tau \mid \text{IsType}(\tau, f) \in \mathcal{R}_{fun}\} \\ \mathcal{U}.\Sigma &\leftarrow \{c \mid \text{HasProperty}(c, f) \in \mathcal{R}_{con}\} \end{aligned} \tag{3}$$

(2) Action II: Mapping $\mathcal{R}_{spa}$.

This action classifies the points into specific spatial roles within the URM. Based on the spatial incidence relations ($\mathcal{R}_{spa}$), points are mapped to the point set $\mathbf{P}_{on}$, intersection points set between function and axes $\mathbf{P}_x, \mathbf{P}_y$, and intersection points set between different functions $\mathbf{P}_{int}$ according to the following logic:

$$\begin{aligned} p \in \mathbf{P}_{on} &\iff \text{IsOn}(p, f) \\ p \in \mathbf{P}_x &\iff p \in \mathbf{P}_{on} \wedge (p.y = 0 \vee \text{IsOn}(p, \text{Axis}_X)) \\ p \in \mathbf{P}_y &\iff p \in \mathbf{P}_{on} \wedge (p.x = 0 \vee \text{IsOn}(p, \text{Axis}_Y)) \\ p \in \mathbf{P}_{int} &\iff p \in \mathbf{P}_{on} \wedge \exists g \neq f : \text{IsOn}(p, g) \end{aligned} \tag{4}$$

(3) Action III: Mapping $\mathcal{R}_{num}$.

Finally, the system anchors explicit quantitative values ($\mathcal{R}_{num}$) to the established structure. Coordinate values are bound to points in $\mathcal{U}$, and specific parameter assignments are injected into the algebraic form within Σ.

$$\text{Attr}(obj) \leftarrow v, \quad \forall \langle obj, v \rangle \in \mathcal{R}_{num} \tag{5}$$

5. Symbolic solver

This section introduces the decoupled inference strategy (DIS) and DIS-enabled symbolic solver to address text-diagram function problems. The procedure, outlined as Procedure IV, is first presented, followed by a detailed explanation of its two core components: the procedural inference layer (responsible for strategic planning of the inference process) and the meta-model inference layer (dedicated to type-specific mathematical execution). These layers form the foundation of the decoupled inference strategy.

5.1. Overview of decoupled inference strategy

Assume the problem recognition stage converts multimodal input into URM instances. The symbolic solver stage then processes these in-

Procedure III: Filling URM with extracted relations.

Input: Relation Set $\mathcal{R}_{all} = \mathcal{R}_T \cup \mathcal{R}_D$
Output: Set of URM instances $\mathbf{\Phi}$

$\mathbf{\Phi} \leftarrow \emptyset$;
foreach *function piece* f *in* $\mathcal{R}_{all}$ **do**
  ▷ **Action I: Mapping** $\mathcal{R}_{fun}$ **and** $\mathcal{R}_{con}$;
  Initialize $\mathcal{U}.\Sigma$ with type and constraint relations from $\mathcal{R}_{fun}$ and $\mathcal{R}_{con}$;
  ▷ **Action II: Mapping** $\mathcal{R}_{spa}$;
  foreach *point* p **do**
    if $IsOn(p, f) \in \mathcal{R}_{spa}$ **then**
      $\mathcal{U}.\mathbf{P}_{on}$.add($p$);
      if $p.y = 0$ ***or*** $IsOn(p, Axis_X)$ **then**
        $\mathcal{U}.\mathbf{P}_{x}$.add($p$);
      if $p.x = 0$ ***or*** $IsOn(p, Axis_Y)$ **then**
        $\mathcal{U}.\mathbf{P}_{y}$.add($p$);
      if $\exists g \neq f$: $IsOn(p, g)$ **then**
        $\mathcal{U}.\mathbf{P}_{int}$.add($p$);
  ▷ **Action III: Mapping** $\mathcal{R}_{num}$;
  Bind values to points or parameters;
  $\mathbf{\Phi}$.add($\mathcal{U}$);
return $\mathbf{\Phi}$;

stances, denoted as $\Phi = \{\mathcal{U}_1, \mathcal{U}_2, \dots, \mathcal{U}_n\}$, where each $\mathcal{U}_i$ corresponds to a distinct function object derived from the problem.

The primary goal is to infer the final solution S from Φ. However, creating a unified solver for diverse text-diagram function problems presents a fundamental contradiction: reasoning logic (e.g., "find intersection") is universal, while computational operations (e.g., solving linear vs. quadratic systems) are highly type-specific. To address this, we propose a decoupled inference strategy (DIS). This strategy splits the solver into two layers: a function meta-model inference layer for type-specific mathematical execution, and a procedural inference layer for strategic planning of the inference process.

Procedure IV is proposed based on the DIS.

Procedure IV: DIS-enabled inference process.

Input: URM instances Φ, Goal G
Output: Solution S

IV-1: Inference with Pre-defined Pathway (IPP);
▷ **if** (Φ, G) *matches a pair* $(U_{init}, G) \in \mathbb{K}$ **then**
  Activate inference pathway π;
  foreach $\mathcal{M}_i \in \pi$ **do**
    $\Phi \leftarrow \mathcal{T}(\mathcal{M}_i, \Phi)$;
IV-2: Inference with Exploratory Pathway (IEP);
▷ **else**
  while $\Phi' \neq \Phi$ **do**
    $\Phi' \leftarrow \Phi$;
    foreach $\mathcal{M}_j = \langle I, \mathcal{S}, \mathcal{O}, \mathcal{T} \rangle \in \Lambda$ **do**
      foreach *URM instance* $u \in \Phi$ **do**
        if $\mathcal{S}(u)$ *is True* **then**
          $\Phi \leftarrow \mathcal{T}(\mathcal{M}_j, \Phi')$;
$S \leftarrow$ SolveEquation(Φ);
return S;

5.2. Procedural inference layer

The procedural inference layer acts as the strategic controller, determining the sequence of function meta-model applications. We construct a knowledge base $\mathbb{K}$ that stores pre-defined inference pathways, mapping problem patterns to solving sequences. The layer employs two distinct rules to manage the inference process.

The first rule, *inference with pre-defined pathway* (IPP), is employed for standard problems. Based on an analysis of the six function types, we have pre-identified a set of common problem understood states (in this work, these understood states are instantiated as URM instances) U_{init} and solving goals G. For each valid pair (U_{init}, G), an inference pathway $\pi = [\mathcal{M}_1, \mathcal{M}_2, \dots]$ is constructed. This pathway represents a coherent sequence of function meta-models whose sequential execution transforms the initial state U_{init} into the solving goal state G. These mappings are indexed in the pool as $\mathbb{K} : (U_{init}, G) \rightarrow \pi$. During inference, the solver matches the current problem state Φ and goal G against $\mathbb{K}$; if a match is identified, the associated pathway π is retrieved and executed directly to obtain the solution efficiently.

The second rule, *inference with exploratory pathway* (IEP), is activated for novel or complex problems where the current URM instances or solving goal G fails to match any entry in $\mathcal{K}$. In this scenario, no pre-defined pathway exists to bridge the current understood state to the solving goal. To address this, the solver iteratively traverses and applies all available function meta-models in Λ. When a function meta-model is matched and applied, its internal mathematical operations are executed, and the resulting information is updated into the URM state Φ. The state transition at each iteration k is formalized as:

$$\Phi_{k+1} = \Phi_k \cup \{\text{Apply}(\mathcal{U}, m) \mid \mathcal{U} \in \Phi_k, m \in \Lambda, \mathcal{S}_m(\mathcal{U})\} \tag{6}$$

In this formulation, m denotes a function meta-model in the model pool Λ, and $\mathcal{S}_m(\mathcal{U})$ represents the structural matching predicate, which verifies whether the URM instance $\mathcal{U}$ satisfies the prerequisites of model m. The formal definitions of these components $(m, \Lambda, \mathcal{S})$ are provided in the subsequent Section 5.3.

This iterative cycle continues until the solving goal is satisfied or the state converges ($\Phi_{k+1} = \Phi_k$), ensuring the exhaustive extraction of derivable knowledge from the provided conditions.

5.3. Function meta-model inference layer

The function meta-model encapsulates the involved type-specific operations.

5.3.1. Definition of function meta-model

Definition 7 (Function Meta-Model)**.** A function meta-model is defined as a quadruple $\mathcal{M} = \langle I, \mathcal{S}, \mathcal{O}, \mathcal{T} \rangle$. In this tuple, I serves as the unique identifier. $\mathcal{S}$ denotes the matching structures, defined as a logical predicate on the URM substructures used for pattern matching. $\mathcal{O}$ represents the type-specific operations, encapsulating the distinct algebraic logic required to construct and solve equations for each function type. $\mathcal{T}$ specifies the transformation logic for updating the URM state upon execution.

Formally, the application of a model $\mathcal{M}$ to a URM $\mathcal{U}$ is governed by a matching function:

$$\text{Apply}(\mathcal{U}, \mathcal{M}) = \begin{cases} \mathcal{T}(\mathcal{O}(\mathcal{U})) & \text{if } \mathcal{S}(\mathcal{U}) \text{ is true} \\ \emptyset & \text{otherwise} \end{cases} \tag{7}$$

This design enables a uniform interface where the solver matches models based solely on structural states (e.g., "has two points"), while the internal $\mathcal{O}$ dynamically adapts to the specific function type (e.g., linear vs. quadratic).

5.3.2. The pool of function meta-models

This work constructs a pool of function meta-models, denoted as Λ, to cover the diverse mathematical operations required for TDF problems. The meta-model pool Λ encompasses 27 core computational models. This pool is organized into three subsets based on their functional utility: $\Lambda = \Lambda_1 \cup \Lambda_2 \cup \Lambda_3$. The specific meaning of each subsets are detailed in the subsequent paragraphs.

The first category, *algebraic definition models* (Λ_1), is designed to derive the analytic expression of a function from function conditions. A representative model in this category is the point-based determination model, which is triggered when the URM lacks an expression but contains a sufficient number of points $|\mathbf{P}_{on}| \geq N$. The internal operation of this model follows type-specific: for a linear function ($N = 2$), it constructs and solves a system $y = kx + b$; for a quadratic function ($N = 3$), it solves the general form $y = ax^2 + bx + c$. Another key model utilizes the vertex property for quadratic functions, triggered by the presence of a vertex constraint, to construct the efficient vertex form equation $y = a(x - h)^2 + k$.

The second category, *function property models* (Λ_2), focuses on extracting specific function features from a known functional expression. This includes models for calculating intercepts, which set $x = 0$ or $y = 0$ to find intersection points with coordinate axes. For quadratic functions, a specialized model calculates the vertex coordinates $V(-\frac{b}{2a}, \frac{4ac-b^2}{4a})$ directly from the coefficients. These models enable the solver to propagate information from the algebraic domain back to the function domain, enriching the problem state.

The third category, *function interaction models* (Λ_3), manages the reasoning regarding interactions between multiple functions. The primary model in this group is the intersection solver, which is activated when two distinct URMs both possess determined expressions. It constructs an algebraic equation by equating the two expressions, $f_i(x) = f_j(x)$, to find the common roots. This category is essential for solving composite problems where the solution depends on the structural relationship between different function objects.

6. The complete workflow for solving an example using UDfSolver

This section uses an example to illustrate how the proposed UDfSolver framework works. The given problem has a text description and a diagram. The problem text provides the function definition $y = ax^2 + bx + c$, the vertex $V(1, -4)$, and one intersection point between function and x-axis $A(-1, 0)$. The problem diagram visually complements this by displaying the full curve. Specifically, it marks another intersection point between function and x-axis with the coordinate value "3" (without a label) and shows the intersection point between function and y-axis (without label or value).

The entire solving process includes four steps.

Step 1: text understanding.

The UDfSolver initiates the problem recognizance process by parsing and annotating the text with key words and POS, and then uses the S^2F model to acquire relations. The explicit algebraic expression triggers two function definition model, and extracts two function relation $\mathbf{r_1}$: IsType(f_1, QUAD), $\mathbf{r_2}$: Para(f_1, a, b, c). The description "vertex $V(1, -4)$" activates a constraint model and a numeric model, which derives the constraint relation $\mathbf{r_3}$: Vertex(V, f_1), and numeric relation $\mathbf{r_4}$: Coor($V, (1, -4)$). Similarly, "intersects x-axis at $A(-1, 0)$" leads the spatial relation $\mathbf{r_5}$, $\mathbf{r_6}$, $\mathbf{r_7}$ and numeric relation $\mathbf{r_8}$. These steps consolidate the textual information into the relation set $\mathcal{R}_T = \{\mathbf{r_1}, \mathbf{r_2}, \mathbf{r_3}, \mathbf{r_4}, \mathbf{r_5}, \mathbf{r_6}, \mathbf{r_7}, \mathbf{r_8}\}$.

Step 2: diagram understanding.

Concurrently, the procedure of diagram parsing finds a parabola piece, two axes and four points. The feature of opening upwards results in a constraint relation $\mathbf{r_9}$, which implies parameter $a > 0$. The intersection point on the x-axis with label A instantiates spatial relations $\mathbf{r_{10}}$, $\mathbf{r_{11}}$, $\mathbf{r_{12}}$. Another unlabeled intersection point on the x-axis at coordinate 3 is identified as object P_2 and acquires spatial relations $\mathbf{r_{13}}$, $\mathbf{r_{14}}$, $\mathbf{r_{15}}$ and numeric relation $\mathbf{r_{16}}$. Additionally, the intersection with the y-axis is identified as object P_1, and produces the relation $\mathbf{r_{17}}$, $\mathbf{r_{18}}$, $\mathbf{r_{19}}$. These form the diagram relation set $\mathcal{R}_D = \{\mathbf{r_9}, \mathbf{r_{10}}, \mathbf{r_{11}}, \mathbf{r_{12}}, \mathbf{r_{13}}, \mathbf{r_{14}}, \mathbf{r_{15}}, \mathbf{r_{16}}, \mathbf{r_{17}}, \mathbf{r_{18}}, \mathbf{r_{19}}\}$.

Step 3: filling URM with extracted relations.

In the final step, the procedure unifies $\mathcal{R}_T$ and $\mathcal{R}_D$ into the URM structure. Action I initializes the function type set $\Sigma = \{\text{QUAD}\}$ and maps the parameter property and vertex constraint. Action II aggregates the explicit points V, A and the detected points P_1, P_2 in diagram into the set $\mathbf{P}_{on}$. Besides, points A, P_1, P_2 are further added to $\mathbf{P}_x$ and $\mathbf{P}_y$ according to their related spatial relations. Action III anchors the numerical coordinates and symbolic constraints to these objects. The final instantiated state is represented as:

$$\mathcal{U} = \left\langle \begin{array}{l} \Sigma = \{\text{QUAD}, \text{Par}(a, b, c), \text{Vertex}(V)\}, \\ \mathbf{P}_{on} = \{A(-1, 0), P_1, V(1, -4), P_2(3, 0)\}, \\ \mathbf{P}_x = \{A, P_2\}, \quad \mathbf{P}_y = \{P_1\}, \quad \mathbf{P}_{int} = \emptyset \end{array} \right\rangle \tag{8}$$

This structured representation explicitly encodes the semantic roles of all primitives and algebraic constraints, providing a uniform understood state for the symbolic solver.

Step 4: symbolic solver

The solver initiates with the URM instance $\mathcal{U}$, which contains the quadratic type signature and property constraint Σ, point set $\mathbf{P}_{on} = \{A(-1, 0), P_1, V(1, -4), P_2(3, 0)\}$, x-intersect point set $\mathbf{P}_x = \{A, P_2\}$, and y-intersect point set $\mathbf{P}_y = \{P_1\}$.

The procedural inference layer matched $\mathcal{U}$ and solving goal against the knowledge base $\mathbb{K}$. It identifies the input $\mathcal{U}$ as a known pattern ("quadratic with vertex and point") in $\mathbb{K}$ and retrieves the corresponding pre-defined pathway: $\pi = [\mathcal{M}_1, \mathcal{M}_2]$.

Following the pathway, the solver applies function meta-model $\mathcal{M}_1$ using $V(1, -4)$, constructing the template equation: $y = a(x - 1)^2 - 4$ according to the type signature Σ and updating into Φ. Then, the solver applies function meta-model $\mathcal{M}_1$ using the $A(-1, 0)$ to generate the equation $0 = a * (-1 - 1)^2 - 4$. Finally, the solver solves all the equations and gets the expression $y = (x - 1)^2 - 4$.

7. Experiments

7.1. Experimental setup

Three popularly used datasets are chosen to evaluate the proposed algorithm and they are: TDF2K, MultiMath-Func and CMM-Func. Table 5 lists the prepared three datasets and the function type distribution about them.

- **TDF2K**: A manually collected dataset comprising 2,049 text-diagram function (TDF) problems, systematically covering six fundamental function types. This dataset integrates problems from two primary sources: (1) the TnD1K dataset originally prepared by Yu et al. (2022), which consists of 1,000 linear function problems collected from middle school textbooks published by People's Education Press (PEP), Beijing Normal University Press (BNU), and Jiangsu Phoneix Science Press (JPS), as well as entrance examination papers from major Chinese cities during 2016-2020; and (2) an additional 1,049 problems newly collected from supplementary educational materials, including "Yiben for Middle School Mathematics: Functions" published by Hunan Education Press and "Zuoyebang: Functions in Middle School Mathematics" published by Xi'an Publishing House. The dataset is designed to provide a comprehensive representation of TDF problem diversity, with 60.5% text-only problems and 39.5% text-diagram problems.
- **MultiMath-Func**: A sub-dataset of MultiMath-300K (Peng et al., 2024), which is a comprehensive multimodal mathematical dataset with 298,670 K-12 problems featuring bilingual descriptions and step-by-step solutions. This subset covers all six function types and serves as a standardized benchmark for assessing multimodal reasoning capabilities, with particular emphasis on problems requiring

Table 4
Performance comparison of UDfSolver against baseline algorithms (Yu-Relation, GPT-4V, MultiMath-7B, Qwen3-VL-8B) across six function types.

Dataset	Methods	Accuracy (%)						
		All	Linear	Quadratic	Inverse	Exponential	Logarithmic	Power
TDF2K	Yu-Relation	60.3	60.3	/	/	/	/	/
	GPT-4V	59.0	64.2	55.5	56.8	56.1	54.9	55.8
	MultiMath-7B	62.2	67.3	59.2	61.0	60.5	59.8	60.1
	Qwen3-VL-8B	60.3	62.5	57.0	63.7	59.2	58.5	51.6
	UDfSolver	**75.6**	**80.0**	**69.5**	**72.1**	**71.5**	**70.3**	**70.9**
MultiMath-Func	Yu-Relation	57.5	57.5	/	/	/	/	/
	GPT-4V	61.8	61.3	62.8	60.8	61.5	60.2	61.1
	MultiMath-7B	67.8	69.5	64.4	63.0	65.2	63.9	64.8
	Qwen3-VL-8B	63.9	64.9	63.6	65.2	61.4	62.7	63.8
	UDfSolver	**78.4**	**80.4**	**77.0**	**76.5**	**78.2**	**77.5**	**78.0**
CMM-Func	Yu-Relation	58.5	58.5	/	/	/	/	/
	GPT-4V	62.5	68.1	60.2	59.7	61.2	60.5	61.0
	MultiMath-7B	66.8	71.5	64.3	64.0	65.5	64.8	65.1
	Qwen3-VL-8B	64.2	68.5	62.0	62.8	60.4	59.5	58.6
	UDfSolver	**73.4**	**78.2**	**71.5**	**70.9**	**72.3**	**71.8**	**72.5**

Note: The 'All' column denotes the micro-average accuracy calculated across the entire dataset. The 'All' score for Yu-Relation represents accuracy calculated only on the linear problems, as the method is not designed for non-linear functions.

Table 5
Distribution of three benchmark datasets across function type.

Dataset	Linear	Quad.	Inv. P.	Exp.	Log.	Pow.	Total
TDF2K	1,139	638	105	59	39	69	**2,049**
MultiMath-Func	1,929	2,164	62	321	437	175	**5,088**
CMM-Func	1,327	1,161	213	258	195	163	**3,317**

both problem understanding and mathematical reasoning across diverse educational levels.

- **CMM-Func**: A function-focused subset extracted from CMM-Math (Liu et al., 2025), a Chinese multimodal mathematics dataset with over 28,000 problems spanning 12 grade levels. This work filtered the data item that focuses exclusively on function-related problems spanning the selected six function types (Linear, Quadratic, Inverse Proportional, Exponential, Logarithmic, and Power functions). Therefore the CMM-Func dataset is particularly suitable for evaluating the performance of function problem solving methods.

The proposed algorithm will be compared with the following four state-of-the-art algorithms.

- **Yu-Relation** (Yu et al., 2022): A relation-centric algorithm specifically designed for solving linear piecewise function problems, representing the state-of-the-art in symbolic TDF solvers. This algorithm employs the S^2P (Syntax-Semantics with Period) method for text understanding and the L^2 (Line segment with Labels Pattern) method for diagram understanding, and a symbolic solver to generate and solve mixture relation groups through an equation-function interaction method. However, it is limited to linear function types only and cannot handle non-linear functions such as quadratic, exponential, or logarithmic functions. We test this baseline only on linear function problems to provide a fair comparison within its designed scope, serving as a representative of specialized symbolic solvers.
- **GPT-4V** (OpenAI, 2023): A state-of-the-art Multimodal Large Language Model (MLLM) developed by OpenAI, capable of processing both text and image inputs for mathematical reasoning. GPT-4V demonstrates strong performance across diverse mathematical tasks through its end-to-end learning approach and has become a standard for evaluating multimodal mathematical reasoning capabilities. However, as an MLLM-based method, it may suffer from hallucinations and lack precise handling of rigorous function properties (e.g., domain constraints, monotonicity), which are critical for educational applications. We evaluate GPT-4V using zero-shot prompting with carefully designed instructions, representing the capabilities of general-purpose closed-source MLLMs.
- **MultiMath-7B** (Peng et al., 2024): A domain-specific MLLM optimized for mathematical reasoning, built upon DeepSeekMath-RL-7B with visual capabilities through a four-stage training process including vision-language alignment, visual instruction-tuning, mathematical instruction-tuning with chain-of-thought reasoning, and process-supervised reinforcement learning. Among open-source models, MultiMath-7B achieves state-of-the-art performance on multimodal mathematical benchmarks while maintaining strong text-only reasoning capabilities. This baseline represents the leading open-source math-specialized MLLMs, serving as a strong comparison point for evaluating domain-adapted multimodal reasoning approaches.
- **Qwen3-VL-8B** (Bai et al., 2025): The latest iteration of the Qwen-VL series developed by Alibaba Cloud, representing the cutting edge of open-source general-purpose MLLMs. Qwen3-VL-8B features significant architectural advancements in dynamic resolution and visual perception, enabling it to comprehend complex charts, geometric diagrams, and dense mathematical texts with remarkable accuracy.

7.2. Overall performance evaluation

The overall performance of our algorithm is summarized in Table 4, demonstrating consistent and significant superiority across three diverse benchmarks and six function types. We analyze the results from multiple perspectives to provide a comprehensive understanding of the proposed method's advantages.

Overall Performance Comparison. UDfSolver achieves leading accuracies of 75.6% on TDF2K, 78.4% on MultiMath-Func, and 73.4% on CMM-Func, substantially outperforming all baselines. Specifically, compared to the strongest baseline MultiMath-7B, our method achieves absolute improvements of +13.4%, +10.6%, and +6.6% respectively. Against GPT-4V, the improvements are even more pronounced at +16.6%, +16.6%, and +10.9%. These consistent gains across datasets with different characteristics (TDF2K: middle school curricula; MultiMath-Func: K-12 diverse problems; CMM-Func: Chinese multimodal contexts) validate the robustness and generalizability of our decoupled inference approach.

Baseline Performance Analysis. Among the baselines, MultiMath-7B consistently outperforms GPT-4V across all three datasets, demon-

strating the advantages of domain-specific training for mathematical reasoning. MultiMath-7B achieves 62.2%, 67.8%, and 66.8% on the three datasets respectively, surpassing GPT-4V by 3.2%, 6.0%, and 4.3%. This performance gap highlights that specialized mathematical training provides substantial benefits over general-purpose multimodal capabilities. Yu-Relation, despite being limited to linear functions only, achieves competitive performance on linear problems (60.3% on TDF2K, 57.5% on MultiMath-Func, 58.5% on CMM-Func), demonstrating the effectiveness of symbolic reasoning approaches within their designed scope. However, its inability to handle non-linear functions fundamentally limits its applicability to comprehensive TDF problem solving.

Function Type Performance Analysis. Our method demonstrates strong and consistent performance across all six function types, with particularly notable advantages in handling complex function categories. On linear functions, UDfSolver achieves the highest accuracies (80.0%, 80.4%, 78.2%), representing absolute improvements of +12.7%, +10.9%, and +6.7% over MultiMath-7B. For non-linear functions, the advantages are even more substantial. On quadratic functions, our method achieves 69.5%, 77.0%, and 71.5%, outperforming MultiMath-7B by +10.3%, +12.6%, and +7.2%. For exponential functions, the improvements are +11.0%, +13.0%, and +6.8%. These consistent gains across function types indicate that our Uniform Representation Model and Decoupled Inference Strategy effectively capture the mathematical essence of diverse function problems, rather than relying on type-specific heuristics.

Performance Consistency and Stability. A critical advantage of our method is its consistent performance across different function types. While MLLM-based baselines exhibit significant performance fluctuations across function types (GPT-4V's range: 54.9%-64.2% on TDF2K; MultiMath-7B's range: 59.2%-67.3%), UDfSolver maintains more stable performance with narrower ranges (69.5%-80.0% on TDF2K, 76.5%-80.4% on MultiMath-Func, 70.9%-78.2% on CMM-Func). This stability indicates that our method does not exhibit strong biases toward particular function types, suggesting better generalization capabilities and more reliable performance in diverse application scenarios.

Cross-Dataset Generalization. Examining performance trends across datasets reveals interesting insights into generalization capabilities. MultiMath-7B shows the largest performance gain on MultiMath-Func (67.8%), which aligns with its training data distribution, but exhibits relatively weaker performance on CMM-Func (66.8%), which features different problem formats and Chinese contexts. In contrast, UDfSolver maintains more consistent relative performance across datasets (75.6%, 78.4%, 73.4%), suggesting that our symbolic reasoning approach is less susceptible to dataset-specific biases and better captures fundamental mathematical principles that transfer across different problem contexts.

Margin of Improvement. The performance improvements of UDfSolver over the strongest baseline (MultiMath-7B) are substantial and statistically significant. The relative improvements of 21.5% (TDF2K), 15.6% (MultiMath-Func), and 9.9% (CMM-Func) represent major advances in TDF problem-solving capabilities. These gains are particularly impressive considering that MultiMath-7B represents the state-of-the-art among domain-specific math MLLMs with specialized training on mathematical reasoning tasks. The consistent superiority across all evaluation dimensions–overall accuracy, individual function types, and different datasets–demonstrates the fundamental advantages of our decoupled symbolic reasoning approach over end-to-end neural methods.

7.3. Case study analysis

To provide a deeper insight into the problem-solving mechanisms of UDfSolver compared to baseline methods, we conducted a qualitative analysis on three representative cases. These cases were selected to evaluate the system's performance across three critical dimensions of TDF problem solving: (1) *multimodal alignment*, testing the ability to integrate visual and textual cues; (2) *cross-type inference*, evaluating the logic required to solve interactions between heterogeneous function types; and (3) *property constraint reasoning*, assessing the rigor in handling definition-based parameter problems. Table 6 details the problem descriptions, the reasoning processes of different models, and the final outcomes.

7.3.1. Multimodal alignment (Case 1)

Case 1 illustrates the challenge of aligning information from different modalities. The problem provides the vertex $V(1,-2)$ in the diagram and a passing point $P(3,2)$ in the text description. Traditional methods like Yu-Relation fail here because their representation schema is rigid, treating points merely as isolated coordinates without semantic roles. While GPT-4V successfully recognized the pattern, MultiMath-7B failed due to a calculation error in the final step, a common pitfall in end-to-end neural models. In contrast, UDfSolver demonstrated robust multimodal alignment. The URM explicitly categorized V as a *Vertex* object within the quadratic function schema. This triggered the specific "Vertex Form" meta-model ($y = a(x-h)^2 + k$), which is structurally more efficient than the general form, thereby simplifying the subsequent symbolic computation and guaranteeing the correct solution.

7.3.2. Cross-type inference (Case 2)

Case 2 examines the system's ability to handle interactions between heterogeneous function types–specifically, the intersection of a quadratic function and a linear function. This case highlights the limitations of specialized solvers like Yu-Relation, which lacks the ontology to represent non-linear objects. Both large language models (GPT-4V and MultiMath-7B) solved this problem correctly, leveraging their broad pre-trained knowledge of algebra. However, UDfSolver achieved the same success through a more explainable mechanism. The DIS activated the relational interaction model (Λ_3), which constructs a joint equation system regardless of the underlying function types. This modularity ensures that the correct intersection logic is consistently applied, whether the functions are linear-linear, linear-quadratic, or other combinations.

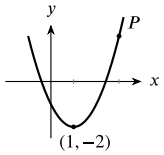
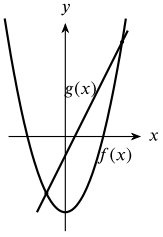
7.3.3. Property constraint reasoning (Case 3)

Case 3 presents the most significant challenge: reasoning about property constraints involving parameters. The task is to determine values for m and n such that $y = (m+1)x^{2-|m|} + n - 4$ becomes a linear function. This requires rigorous adherence to mathematical definitions, specifically the non-zero coefficient constraint ($m + 1 \neq 0$). As shown in Table 6, both GPT-4V and MultiMath-7B failed this test. GPT-4V correctly identified the degree constraint ($2 - |m| = 1$) but neglected the coefficient constraint, leading to an extraneous solution ($m = -1$). MultiMath-7B hallucinated an unnecessary constraint on the constant term ($n = 4$), restricting the solution space incorrectly. UDfSolver, however, successfully avoided these traps. Its knowledge base encodes the strict definition of function types as logical rules. When the "Linear Function" target was identified, the system automatically enforced both necessary conditions–exponent equality and non-zero coefficient–demonstrating the superiority of symbolic constraints over probabilistic generation in tasks requiring rigorous logic.

7.4. Component ablation study

To quantify the impact of each UDfSolver module, we conducted a component-level evaluation on the TDF2K dataset. To ensure a fair comparison across all six function types, all configurations utilize our proposed perception modules (S^2F and F^2) to extract relations. As summarized in Table 7, removing either the URM or the DIS results in a substantial degradation in overall accuracy.

Table 6
Demonstrating UDfSolver's robust inference advantage over baselines' limitations through three TDF inference cases.

Case Type	Problem Description	Method	Reasoning Process & Outcome
Case 1	**Text**: "A quadratic function has vertex V and passes through point $P(3,2)$. Find its expression." **Diagram**: 	**Yu-Relation**	**Failed.** The relation-centric model could not map the visual "vertex" feature to the algebraic quadratic form, treating it as a general point.
		GPT-4V	**Correct.** "Vertex is $(1,-2)$. Form is $y = a(x-1)^2 - 2$. Plug in $(3,2)$: $2 = a(2)^2 - 2 \rightarrow 4 = 4a \rightarrow a = 1$." Successful pattern recognition.
		MultiMath-7B	**Incorrect.** Correctly identified the vertex form $y = a(x-1)^2 - 2$ but committed an arithmetic error during expansion ($y = x^2 - 2x - 2$).
		UDfSolver	**Correct.** S^2F models extracted $\mathcal{R}_T$ including vertex and point-on relations. F^2 model confirmed $a > 0, b < 0, c < 0$. URM: $\mathcal{U} = \langle \Sigma = \{\text{QUAD}, \text{Par}(a), \text{Vertex}(V)\}, \mathbf{P}_{on} = \{V(1,-2), P(3,2)\}\rangle$. DIS-enabled symbolic solver: Matched pattern "quad+vertex+point" in $\mathbb{K}$, triggered $\pi = [\mathcal{M}_{\text{vertex-form}}, \mathcal{M}_{\text{point}}]$. Meta-model $\mathcal{M}_{\text{vertex-form}}$ constructed $y = a(x-1)^2 - 2$ using vertex semantics; $\mathcal{M}_{\text{point-sub}}$ substituted $P(3,2)$ to obtain $a = 1$.
Case 2	**Text**: "Find intersection points of $f(x) = x^2 - 4$ and $g(x) = 2x - 1$." **Diagram**: 	**Yu-Relation**	**Failed.** Limited by its linear-only ontology, the system could not represent the quadratic function or the intersection relation between heterogeneous types.
		GPT-4V	**Correct.** "Set $x^2 - 4 = 2x - 1$. Rearrange to $x^2 - 2x - 3 = 0$. Factors to $(x-3)(x+1) = 0$. Points are $(3,5)$ and $(-1,-3)$."
		MultiMath-7B	**Correct.** Verified the visual intersection points against the text equations, confirming that $(3,5)$ and $(-1,-3)$ satisfy both functions.
		UDfSolver	**Correct.** S^2F models parsed two function expressions. F^2 models identified curve types. URM: Two instances $\mathcal{U}_f = \langle \Sigma = \{\text{QUAD}, \text{Expr}(y = x^2 - 4)\}\rangle$ and $\mathcal{U}_g = \langle \Sigma = \{\text{LIN}, \text{Expr}(y = 2x - 1)\}\rangle$. DIS-enabled symbolic solver: Matched "two-func+intersection" in $\mathbb{K}$, activated $\mathcal{M}_{\text{int}} \in \Lambda_3$. Meta-model constructed joint equation $x^2 - 4 = 2x - 1$, solved to get $x \in \{-1, 3\}$, and updated $\mathbf{P}_{int} = \{(-1,-3),(3,5)\}$. Type-specific operation ensures consistent logic across different function types.
Case 3	**Text**: "Given $y = (m+1)x^{2-\lvert m\rvert} + n - 4$, if y is a linear function, what values of m, n?"	**Yu-Relation**	**Failed.** Unable to parse the parameter expression embedded in the exponent.
		GPT-4V	**Incorrect.** "Exponent must be 1, so $2 - \lvert m\rvert = 1 \implies \lvert m\rvert = 1 \implies m = \pm 1$." Failed to check the coefficient constraint ($m + 1 \neq 0$), yielding an invalid solution $m = -1$.
		MultiMath-7B	**Incorrect.** Correctly identified exponent constraint but hallucinated a constraint for n ($n - 4 = 0$), incorrectly restricting the constant term.
		UDfSolver	**Correct.** S^2F method extracted parametric form with target type LIN. URM: $\mathcal{U} = \langle \Sigma = \{\text{LIN}, \text{Par}(m,n), \text{Expr}(y = (m+1)x^{2-\lvert m\rvert} + n - 4)\}\rangle$. DIS-enabled symbolic solver: Match "definition-constraint" in $\mathbb{K}$, activated $\mathcal{M}_{\text{linear-def}} \in \Lambda_1$ imposed dual constraints: (1): $2 - \lvert m\rvert = 1 \Rightarrow m \in \{-1, 1\}$; (2) $m + 1 \neq 0 \Rightarrow m \neq -1$; (3) $n - 4 \neq 0 \Rightarrow n \neq 4$. Generate answer $m = 1, n \neq 4$. Symbolic constraint ensures inference rigorous adherence to mathematical definitions.

Specifically, substituting the URM with the traditional relation set (Yu et al., 2022) (- *w/o URM*) led to a performance drop of 10.4% (from 75.6% to 65.2%). In this configuration, the DIS-enabled solver is matched against the unstructured relation set, limiting the solver's ability to access overall function properties. Furthermore, replacing the DIS with an extended general solver (- *w/o DIS*) also caused an accuracy decrease of 11.8% (from 75.6% to 63.8%). Although this general solver (Yu et al., 2022) is adjusted to access information from the URM, it processes all equations without the FMM and decoupled architecture. Consequently, the solver frequently falls into inference conflicts due to the misapplication of type-specific knowledge, such as incorrectly applying quadratic vertex model to exponential function. It is noteworthy that the removal of both URM and DIS (- *w/o URM & DIS*) achieves only 57.4% accuracy. The individual effect of URM or DIS yields small improvements. However, their integration achieves 75.6% accuracy. Together, these quantitative results confirm that both URM and DIS are indispensable, and their synergy is the primary impact of the UDfSolver's superior performance.

7.5. Difficulty stratification analysis

To fully characterize the robustness of the proposed method across varying complexity levels, we conducted a difficulty stratification analysis on the TDF2K dataset. For difficulty classification, we use an objec-

Table 7
Results of the component ablation study on the TDF2K dataset.

Method	Accuracy (%)	Δ (%)
UDfSolver	**75.6**	**–**
- w/o URM	65.2	–10.4
- w/o DIS	63.8	–11.8
- w/o URM & DIS	57.4	–18.2

Note. - *w/o URM* means replacing the URM with the traditional relation set. - *w/o DIS* means replacing the DIS-enabled solver with an extended general solver. - *w/o URM & DIS* removes both URM and DIS-enabled solver, replacing with the relation set and the extended general solver.

Table 8
Accuracy comparison across different difficulty levels on the TDF2K dataset.

Method	Easy (%)	Hard (%)	Accuracy Drop (Δ)
Yu-Relation	67.5	46.1	↓ 22.9
GPT-4V	81.5	22.4	↓ 59.1
MultiMath-7B	84.8	25.4	↓ 59.4
UDfSolver	**88.2**	**55.1**	↓ **33.1**

Note. Yu-Relation was evaluated exclusively on the linear function subset due to its algorithmic constraints.

tive, LLM-as-a-Judge metric. Specifically, three MLLMs were prompted to independently assess the complexity of each problem and directly classify it as either “easy” or “hard”. The final difficulty label was then determined via a majority voting mechanism.

In the TDF2K dataset, the distribution of easy and hard problems is 61.93% and 38.07%, respectively. Table 8 presents the performance comparison between UDfSolver and all baseline algorithms across these two difficulty levels. As problem complexity increases from easy to hard, the performance of MLLM baselines (e.g., GPT-4V and MultiMath-7B) degrades precipitously, dropping by up to 59.4%. While the traditional symbolic method (Yu-Relation) exhibits a flatter drop, its overall capability is also quite low. In contrast, UDfSolver not only achieves superior accuracy on easy problems but also maintains a significantly slower decline compared to MLLM baselines, sustaining robust performance on hard problems. This demonstrates that while end-to-end neural models struggle with complex constraints, our decoupled symbolic reasoning framework guarantees both high capacity and reliability.

7.6. Performance and error analysis

As previously mentioned, the actual input to UDfSolver and problem understanding module is assumed to be vector-version diagram, and the transition from image version to vector version is handled by existing diagram parsing tools. Therefore, to quantitatively isolate the solving ability of the UDfSolver from upstream diagram parsing errors, we conducted a ground truth analysis on a manually annotated subset of 360 representative cases from the TDF2K dataset. Specifically, we compared the performance under two testing scenarios. In the first scenario, the input is raw image version diagram. UDfSolver processes it through the automated parsing pipeline. In the second scenario, UDfSolver is directly inputted with accurately annotated vector-version diagram and structured text tokens. Under this ground truth setting, the UDfSolver achieved accuracy of 87.2%, compared to the original accuracy of 75.0% on the same subset, as shown in Table 9. This contrast demonstrates that the current performance bottleneck lies partially in the multimodal parsing process, rather than the symbolic reasoning process.

Table 9
Ground truth analysis on a manually annotated 360-case subset of TDF2K.

Method	Accuracy	Accuracy Gap (Δ)
UDfSolver	75.0%	–
UDfSolver (GT)	**87.2%**	↑ **12.2%**

Note. GT refers to the use of ground truth parsing results, rather than those generated by the parser.

Meanwhile, We further analyze the failure modes of UDfSolver to identify the underlying principles causing these errors. The limitations are categorized into three primary types.

(1) URM instantiation errors.

The current S^2F models rely on explicit syntactic pattern matching. However, some problems involve implicit functional concepts or physical scenarios (e.g., “uniform motion”). These phrases cannot be directly mapped to function relations without an intermediate common-sense semantic understanding (e.g., translating “uniform motion” to a function definition relation and parameter constraint). The absence of this semantic bridging mechanism causes the system to miss critical constraints and function relations during URM instantiation.

(2) Diagram parsing errors.

The F^2 models rely on global spatial properties to identify function types. In complex diagrams, dense curve segments, curve intersections or text labels can significantly increase the difficulty of accurately parsing and identifying key geometric features. This complex leads to the extraction of flawed feature vectors (V_{fea}), triggering incorrect F^2 templates and injecting erroneous attributes into the URM.

(3) Symbolic solving limitations.

The DIS engine fails when the formulated algebraic system requires specific heuristic manipulations or inequality solving that exceeds the predefined meta-model pool (Λ). For instance, determining the range of a parameter based on geometric constraints (e.g., requiring intersections in a specific quadrant) require translating to solving systems of non-linear inequalities. Additionally, calculating the distance between intersection points of a line and a parabola often requires applying Vieta’s formulas to rewrite symmetric polynomials. If these specific algebraic identities or inequality resolution strategies are not explicitly defined in Λ, the symbolic solver cannot proceed.

8. Discussion

8.1. Evaluation criteria for UDfSolver in education

While end-to-end accuracy is the conventional metric for evaluating solving ability, intelligent problem solvers intended for education require broader evaluation criteria. It is necessary to explicitly state that the proposed UDfSolver focuses on the algorithmic framework for solving middle-school TDF problems, rather than directly evaluating student comprehension or pedagogical effectiveness. However, as this algorithm aims to provide the core technological engine for educational services, it possesses potential for teaching applications. Therefore, drawing inspiration from existing criteria for educational knowledge models (Nawaz, 2024; Nguyen et al., 2018), we propose the evaluation criteria for mathematical problem solving system to assess and validate their potential effectiveness in educational scenarios.

The proposed evaluation criteria encompasses four dimensions. First, final answer accuracy remains the fundamental baseline to ensure the system can derive correct answers. Second, explainability requires that the system’s outputs and processes are comprehensible to students. Third, transparency of reasoning demands that the system’s inference process is explicit and traceable. Finally, extensibility reflects whether the system can adapt to new problem types without redesigning the entire architecture.

Within these evaluation criteria, the components of UDfSolver demonstrate advantages for educational application. Regarding explainability, the proposed neuro-symbolic framework constructs text and diagram patterns, as well as function meta-models, directly derived from pedagogical knowledge and human mathematical understanding. For transparency of reasoning, the URM explicitly represents intermediate understood states, while the DIS breaks down the reasoning into traceable steps. This allows the system to clearly present the applied meta-models and the generated equations during the solving process. In terms of extensibility, new function types can be integrated into the URM by adding new attribute fields or constraint slots. Furthermore, the DIS can also be extended involves adding new function meta-models. Therefore, UDfSolver establishes a foundation for generating pedagogical feedback and integrating into intelligent tutoring environments.

8.2. Generalization of UDfSolver

The uniformity achieved by UDfSolver represents a significant advancement over prior works that were strictly confined to single function types. This uniformity is possible because the six elementary algebraic functions share common structural properties: they can be characterized by finite intrinsic qualitative attributes, their geometric anchors can be represented by finitely many special points, and their reasoning processes can be built on a common structural basis.

However, it is crucial to delineate the generalization boundaries of the current framework. The proposed uniform representation model cannot be directly applied to other function categories (e.g., trigonometric, piecewise, or composite functions), as these categories possess unique mathematical properties outside the current scope. Trigonometric functions involve periodicity and infinite recurring cycles, which would require the decoupled inference strategy to handle periodic solution sets. Composite functions require a hierarchical, nested representation instead of the flat structure used for elementary functions. Piecewise functions necessitate the explicit encoding of segment intervals and boundary conditions, which the current model does not support.

These additional mathematical properties are finite and well-defined, meaning the framework does not require a complete redesign. Instead, the existing architecture can be extended through targeted modifications. For instance, new attribute fields can be added to the core attribute set of the representation model to explicitly encode periodicity and interval constraints. Furthermore, introducing a nested relational element would allow the system to model composite function hierarchies, enabling one instance to serve as a variable input for another. By implementing these structural upgrades to both the representation model and the inference strategy, the current framework can be systematically extended to solve these broader function types.

Additionally, the current UDfSolver is designed for TDF problems that focus on explicit function knowledge acquisition and inference. It is not intended for TDF problems requiring the extraction of implicit function knowledge from real-world or physical scenarios. Solving such problems necessitates open-ended natural language understanding to bridge the gap between natural language narratives and formal mathematical constraints. To address this limitation in future extensions, the large language models could be employed as semantic parsers to translate implicit real-world descriptions into the explicit function relations.

8.3. Scalability of UDfSolver

To substantiate the scalability of the proposed framework, we characterize the engineering cost required to extend UDfSolver to a new function type. Because of the decoupled inference strategy, extending the system requires no modifications to the core procedural inference layer. The engineering effort is limited, requiring only the addition of a small number of specific text templates, visual templates, and function meta-models. This modular design facilitates high scalability with minimal manual intervention.

Regarding the manual expert effort associated with constructing these rule libraries, AI-based information extraction presents a promising direction for future exploration. It is anticipated that this manual effort could be substantially mitigated in future iterations through neuro-symbolic integration. Recent advancements suggest that large language models (LLMs) have the potential to translate informal mathematical statements into formal specifications (Wu et al., 2022; Yu et al., 2025; Zhang et al., 2024b) and to autonomously induce open rules and concept structures from text corpora (Cui & Chen, 2021). Building upon these capabilities, the scalability of UDfSolver might be further enhanced by adopting a neuro-symbolic closed-loop paradigm. In such an envisioned approach, LLMs could act as pattern miners to automatically propose candidate S^2F and F^2 rules from raw datasets. These neural-generated candidates would subsequently be verified and filtered by symbolic mechanisms (e.g., variable consistency checks and geometric feature stability) before being admitted into the formal libraries. This potential synergy aims to leverage deep models for scalable pattern discovery while relying on the symbolic layer to maintain logical reliability, which may ultimately help automate the expansion of the framework with reduced expert intervention.

Fundamentally, discovering implicit problem-solving patterns from mathematical problems conceptually aligns with the broader field of knowledge mining. However, while the macro-level goals are similar, applying advanced mining algorithms to our symbolic framework would require significant future research, particularly in defining exactly what mathematical patterns to look for and how to guide the artificial intelligence models effectively. A notable distinction between mathematical problem-solving pattern extraction and conventional knowledge mining is that the latter often focuses on high-frequency co-occurrence patterns. In contrast, certain mathematical patterns may appear with extremely low frequency yet play a decisive, critical role in solving specific classes of problems. Enabling models to capture these low-frequency but highly critical patterns poses a major challenge. Addressing this challenge represents a key focus for our future research, as we explore ways to realize the aforementioned neuro-symbolic integration.

9. Conclusions and future work

This paper has introduced UDfSolver, a knowledge-driven system that effectively addresses the long-standing challenge of solving diverse Text-Diagram Function (TDF) problems. Our core contribution is a novel architecture that achieves two key innovations: (1) a uniform representation model (URM) that abstracts multimodal inputs into a standardized domain ontology, overcoming the knowledge fragmentation of prior methods; and (2) a decoupled inference strategy that separates procedural execution from domain-specific knowledge while maintaining a unified inference framework.

The UDfSolver framework features a function meta-model layer that serves as a formal knowledge base for both declarative and procedural mathematical knowledge, and a procedural inference layer that orchestrates the problem-solving process. Experimental results demonstrate that UDfSolver achieves superior accuracy, scalability, and interpretability compared to both specialized algorithms and multimodal large language models (MLLMs).

Crucially, our work presents a blueprint for developing more capable and trustworthy AI systems. While MLLMs excel at perceptual pattern recognition, they often struggle with the precision and reliability required for multi-step symbolic computation, resulting in unverifiable reasoning and factual errors. Our framework proposes a hybrid intelligence paradigm that synergizes perception-oriented components for knowledge acquisition with a formal symbolic system for logical reasoning. This separation of concerns offers a promising approach to mitigating the inherent opacity and unreliability of end-to-end models in domains requiring rigorous and verifiable solutions, such as mathematics and science.

Looking ahead, we identify three key research directions: (1) expanding the knowledge base to encompass broader function types (e.g.,

trigonometric and composite functions) and exploring deeper neuro-symbolic integration for automated rule discovery; (2) enriching semantic understanding capabilities to handle implicit relation extraction and function application problems embedded in physical scenarios; and (3) developing stronger function-diagram parsing modules to address the primary performance bottleneck. Furthermore, the explicit representations and reasoning traces generated by UDfSolver may support future educational deployment and inspection, bridging the gap between automated solving and interpretable pedagogical tools.

CRediT authorship contribution statement

Feng Xiong: Methodology, Investigation, Writing – original draft, Visualization, Validation, Software; **Xinguo Yu:** Conceptualization, Writing – review & editing, Supervision, Project administration, Funding acquisition; **Litian Huang:** Formal analysis, Visualization, Validation; **Dian Yu:** Data curation, Resources.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is partially supported by the general Project of the National Natural Science Foundation of China (62277022).

Data availability

The authors do not have permission to share data.

Supplementary material

Supplementary material associated with this article can be found, in the online version, at 10.1016/j.eswa.2026.133229

References

Bai, S., Cai, Y., Chen, R. et al. (2025). Qwen3-vl technical report. arXiv:2511.21631.

Cui, W., & Chen, X. (2021). Open rule induction. *Advances in Neural Information Processing Systems*, *34*, 28536–28547.

De, P., Mandal, S., & Bhowmick, P. (2017). Hierarchical vectorization of electrical drawings in document images by connectivity analysis of symbols and super-components. *Pattern Recognition and Image Analysis*, *27* (2), 309–325. https://doi.org/10.1134/S1054661817020079

Gan, W., Yu, X., & Wang, M. (2019). Automatic understanding and formalization of plane geometry proving problems in natural language: A supervised approach. *International Journal on Artificial Intelligence Tools*, *28* (04), 1940003. https://doi.org/10.1142/S0218213019400037

Goel, A. (2025). Langextract. https://github.com/google/langextract. https://doi.org/10.5281/zenodo.17015090

Hatisaru, V., & Erbas, A. K. (2017). Mathematical knowledge for teaching the function concept and student learning outcomes. *International Journal of Science and Mathematics Education*, *15*, 703–722.

Huang, L., Yu, X., Niu, L., & Feng, Z. (2023). Solving algebraic problems with geometry diagrams using syntax-semantics diagram understanding. *Computers, Materials & Continua*, *77* (1) 517.

Huang, Z., Lin, X., Wang, H., Liu, Q., Chen, E., Ma, J., Su, Y., & Tong, W. (2021). DisenQNet: Disentangled representation learning for educational question. In *Proceedings of the 27th ACM SIGKDD Conference on knowledge discovery & data mining* KDD '21 (pp. 696–704). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3447548.3467347. https://doi.org/10.1145/3447548.3467347

Jian, P., Sun, C., Yu, X., He, B., & Xia, M. (2019). An end-to-end algorithm for solving circuit problems. *International Journal of Pattern Recognition and Artificial Intelligence*, *33* (07), 1940004. https://doi.org/10.1142/S0218001419400044

Lin, C.-C., Huang, A. Y. Q., & Lu, O. H. T. (2023). Artificial intelligence in intelligent tutoring systems toward sustainable education: A systematic review. *Smart Learning Environments*, *10* (1), 41. https://doi.org/10.1186/s40561-023-00260-y

Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. In *Proceedings of the IEEE International conference on computer vision* (pp. 2980–2988).

Liu, W., Pan, Q., Zhang, Y., Liu, Z., Wu, J., Zhou, J., Zhou, A., Chen, Q., Jiang, B., & He, L. (2025). CMM-math: A chinese multimodal math dataset to evaluate and enhance the mathematics reasoning of large multimodal models. In *Proceedings of the 33rd ACM International conference on multimedia* MM '25 (pp. 12585–12591). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3746027.3758193

Lu, P., Bansal, H., Xia, T., Liu, J., Li, C., Hajishirzi, H., Cheng, H., Chang, K.-W., Galley, M., & Gao, J. (2024). Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. https://arxiv.org/abs/2310.02255.

Lu, Y., Pian, Y., Chen, P., Meng, Q., & Cao, Y. (2021). Radarmath: An intelligent tutoring system for math education. *Proceedings of the AAAI Conference on Artificial Intelligence*, *35* (18), 16087–16090. https://doi.org/10.1609/aaai.v35i18.18020

Lyu, X., Yu, X., & Peng, R. (2023). Vector relation acquisition and scene knowledge for solving arithmetic word problems. *Journal of King Saud University - Computer and Information Sciences*, *35* (8), 101673. https://doi.org/10.1016/j.jksuci.2023.101673

Nawaz, M. (2024). Ai in stem education: Enhancing problem-solving and critical thinking skills. *AI EDIFY Journal*, *1* (4), 38–48.

Nguyen, H. D., & Do, N. V. (2017). Intelligent problem solver in education for discrete mathematics. *Frontiers in Artificial Intelligence and Applications*, *297*, 21–34.

Nguyen, H. D., Do, N. V., Tran, N. P., & Pham, X. H. (2018). Criteria of a knowledge model for an intelligent problems solver in education. In *2018 10th International conference on knowledge and systems engineering (KSE)* (pp. 288–293). IEEE.

OpenAI (2023). GPT-4V(ision) system card. https://openai.com/research/gpt-4v-system-card.

Peng, R., Yu, X., Yang, C., & Lyu, X. (2025). A scene-attention relation-centric algorithm for solving arithmetic word problems. *Expert Systems with Applications*, *277*, 127197. https://www.sciencedirect.com/science/article/pii/S095741742500819X. https://doi.org/10.1016/j.eswa.2025.127197

Peng, S., Fu, D., Gao, L., Zhong, X., Fu, H., & Tang, Z. (2024). Multimath: Bridging visual and mathematical reasoning for large language models. arXiv:2409.00147.

Sokolowski, A. (2024). Developing students' reasoning in precalculus: Covariational explorations enriched by rates of change and limits. Springer Texts in Education. Cham: Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-66441-0

Wang, P.-Y., Liu, T.-S., Wang, C., Li, Z., Wang, Y., Yan, S., Jia, C., Liu, X.-H., Chen, X., Xu, J., & Yu, Y. (2025). A survey on large language models for mathematical reasoning. *ACM Computing Surveys*, 3786333. https://doi.org/10.1145/3786333

Wilkie, K. J. (2024). Coordinating visual and algebraic reasoning with quadratic functions. *Mathematics Education Research Journal*, *36* (1), 33–69. https://doi.org/10.1007/s13394-022-00426-w

Wu, Y., Jiang, A. Q., Li, W., Rabe, M., Staats, C., Jamnik, M., & Szegedy, C. (2022). Autoformalization with large language models. *Advances in Neural Information Processing Systems*, *35*, 32353–32368.

Xia, J., & Yu, X. (2021). A paradigm of diagram understanding in problem solving. In *2021 IEEE International conference on engineering, technology & education (TALE)* (pp. 531–535). IEEE.

Yin, Y., Liu, Q., Huang, Z., Chen, E., Tong, W., Wang, S., & Su, Y. (2019). QuesNet: A unified representation for heterogeneous test questions. In *Proceedings of the 25th ACM SIGKDD International conference on knowledge discovery & data mining* KDD '19 (pp. 1328–1336). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3292500.3330900

Yu, X., Cheng, W., Yang, C., & Zhang, T. (2026). A theoretical review on solving algebra problems. *Expert Systems with Applications*, *296*, 128789.

Yu, X., Lyu, X., Peng, R., & Shen, J. (2023). Solving arithmetic word problems by synergizing syntax-semantics extractor for explicit relations and neural network miner for implicit relations. *Complex & Intelligent Systems*, *9* (1), 697–717. https://doi.org/10.1007/s40747-022-00828-0

Yu, X., Sun, H., & Sun, C. (2022). A relation-centric algorithm for solving text-diagram function problems. *Journal of King Saud University-Computer and Information Sciences*, *34* (10), 8972–8984.

Yu, X., Wang, M., Gan, W., He, B., & Ye, N. (2019). A framework for solving explicit arithmetic word problems and proving plane geometry theorems. *International Journal of Pattern Recognition and Artificial Intelligence*, *33* (07), 1940005.

Yu, Z., Peng, R., Ding, K., Li, Y., Peng, Z., Liu, M., Zhang, Y., Yuan, Z., Xin, H., Huang, W. et al. (2025). Formalmath: Benchmarking formal mathematical reasoning of large language models. arXiv:2505.02735, .

Zhang, H., Miao, J., Liu, Z., Wesson, I. L., & Shang, J. (2020). Nlpir-parser: Making chinese and english semantic analysis easier and complete. In *15th International conference on the statistical analysis of textual data*.

Zhang, J., Li, Z.-Z., Zhang, M.-L., Yin, F., Liu, C.-L., & Moshfeghi, Y. (2024a). Geoeval: Benchmark for evaluating llms and multi-modal models on geometry problem-solving. In *Findings of the association for computational linguistics: ACL 2024* (pp. 1258–1276).

Zhang, L., Quan, X., & Freitas, A. (2024b). Consistent autoformalization for constructing mathematical libraries. In *Proceedings of the 2024 Conference on empirical methods in natural language processing* (pp. 4020–4033).

Zhang, R., Jiang, D., Zhang, Y., Lin, H., Guo, Z., Qiu, P., Zhou, A., Lu, P., Chang, K.-W., Qiao, Y. et al. (2024c). Mathverse: Does your multi-modal llm truly see the diagrams in visual math problems? In *European conference on computer vision* (pp. 169–186). Springer.